



UroFeed

CLINICAL · V0.1

Technical Architecture Overview

`/* engineering reference */`

An invite-only educational portal for practicing urologists. Aggregates publicly posted tweets from a curated list of accounts and hashtags during major urology congresses, then uses an LLM to summarize the discussion per session, abstract, and congress-day.

This document maps the system as it stands today — the stack, the data model, the ingestion and summarization pipelines, the frontend organization, the operational surfaces, and the trust boundaries.

Contents

21 sections covering system architecture, data model, operational layer, and current technical state.

01	Executive summary	03
02	System architecture overview	04
03	Technology stack	05
04	Database — core domain model	06
05	Database — per-user tier	07
06	Table reference and key constraints	08
07	Authentication and Row-Level Security	09
08	Tweet ingestion pipeline	10
09	Tweet-to-session matcher cascade	11
10	Summarization pipeline	12
11	AI Gateway integration	13
12	Operational layer — cron topology	14
13	Frontend — routes and shell	15
14	Frontend — component organization	16
15	Mobile architecture	17
16	Compose and posting flow	18
17	Digests and email pipeline	19
18	Discover and recommendation system	20
19	Demo account architecture	21
20	Observability and audit	22
21	Technical debt and roadmap	23

01 Executive summary

S01 · 2026-05

UroFeed is a closed, invite-only educational portal for practicing urologists. It aggregates publicly posted tweets from a curated list of accounts and hashtags during major urology congresses (EAU, AUA, SIU, APCCC, ESMO-GU, ASCO-GU, WCE, ICS, ESSM, others), classifies them against an admin-curated program of sessions, and uses an LLM to synthesize per-session, per-abstract, and per-congress-day summaries.

PRODUCT CAPABILITIES AT A GLANCE

INGESTION	X API v2 pluggable adapter; hashtag and per-source pulls; full hierarchy capture (reply / quote / retweet metadata); media URLs; rate-limit aware.
CLASSIFICATION	Seven-step matcher cascade routing tweets to sessions: hashtag → abstract_number → speaker → entity → time_window → LLM → thread propagation.
SUMMARIZATION	Smart-debounced regeneration (≥5 new tweets OR ≥30 min stale) via Lovable AI Gateway; structured tool-call output; session and congress-day granularity.
CURATION	Admin-managed recommendations matrix by specialty; user-managed personal follows; named source groups by cancer area.
PERSONALIZATION	7-step onboarding wizard; topic / specialty / congress / source-bound email digests with smart-debounce; in-app watchlists with topic alerts.
POSTING	Compose-to-X integration with AI tone presets; quote-share from any tweet card; mobile FAB; demo-mode sandbox.
DEMO	Single shared sandboxed account with 15+ pre-populated sources, 3 congresses, 5+ sessions, 3 digests, 5 simulated posts; nightly self-reset.

ARCHITECTURAL PRINCIPLES

SERVICE LAYER ABSTRACTION

All data flows through `src/services/feedService.ts` and `src/services/aiService.ts`. Components never import database or adapter code directly. Backend selection (mock vs api) flips via a single env flag.

SERVER-SIDE SECRETS ONLY

`X_BEARER_TOKEN`, `LOVABLE_API_KEY`, and the service role key live in Lovable Cloud secrets and Postgres vault. They never enter `src/components/**` or any browser-reachable code path.

GLOBAL TWEETS, PER-USER FILTERS

A single ingestion serves all users. The `tweets` table is global with RLS read-all-for-authenticated. User-specific filtering happens at query time via `user_subscribed_*` tables. One source ingested means one source ingested regardless of how many users follow it.

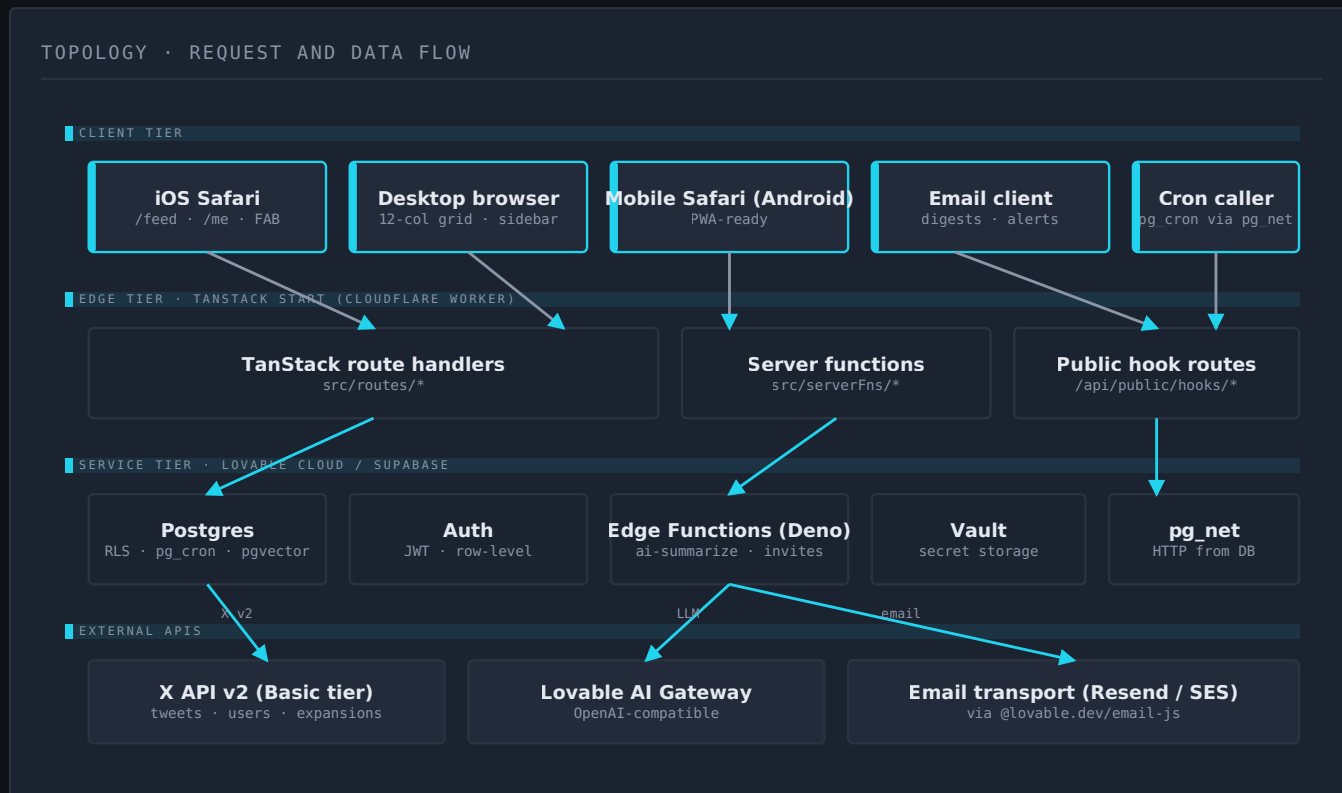
VAULT AS SINGLE SOURCE OF TRUTH

The `X_JOB_SECRET` drift problem was solved structurally: cron-gated routes read the secret via a `SECURITY_DEFINER` RPC backed by Postgres vault. The env variable `process.env.X_JOB_SECRET` no longer exists in the codebase.

02 System architecture overview

S02 · COMPONENT TOPOLOGY

Four tiers, each independently scalable. Client → Edge → Service → External.



TIER RESPONSIBILITIES

CLIENT TIER	React 19 + TanStack Router rendering. iOS Safari is the primary mobile target; desktop browsers get a denser RayStation-style workstation layout. Email clients receive HTML digests rendered server-side. <code>pg_cron</code> acts as a non-human client.
EDGE TIER	TanStack Start runs on Cloudflare Worker runtime. File-based routing with co-located server functions. The <code>/api/public/hooks/*</code> namespace exposes routes callable by <code>pg_cron</code> with shared-secret auth.
SERVICE TIER	Lovable Cloud (Supabase under the hood) provides Postgres with <code>pg_cron</code> + <code>pg_net</code> + <code>pgvector</code> extensions, JWT auth, Vault for secrets, and Deno-runtime Edge Functions for email + AI proxy.
EXTERNAL	X API v2 Basic tier for tweet ingestion (≈10K tweets/month). Lovable AI Gateway for summarization and classification (OpenAI-compatible). Email transport for digest delivery.

WHY THIS SHAPE

Cloudflare Worker runtime gives global edge latency and ~zero cold-start, but constrains the runtime — no Node-only APIs (no `crypto.createHash`, no `fs`). Web Crypto via `crypto.subtle` is the substitute. Supabase as service tier means RLS + `pg_cron` + Vault are first-class, removing the need for a separate scheduler, secret manager, or auth service.

FRONTEND

LAYER	CHOICE	NOTES
Framework	TanStack Start 1.167	File-based routing, server functions, SSR. Cloudflare Worker runtime.
Library	React 19	Suspense, concurrent rendering.
Styling	Tailwind 4 + shadcn/ui	Radix primitives. Dark-only RayStation theme via CSS vars.
Data fetching	@tanstack/react-query 5	Optimistic mutations, cache invalidation by key.
Virtualization	@tanstack/react-virtual	Tweet streams.
Forms	react-hook-form + zod	Schema-validated on server boundary.
PDF export	jsPDF + html2canvas	Direct text rendering (selectable). Lazy-loaded.
Charts	Recharts	LiveSignals, activity sparklines.
Icons	lucide-react	Single source for all glyphs.

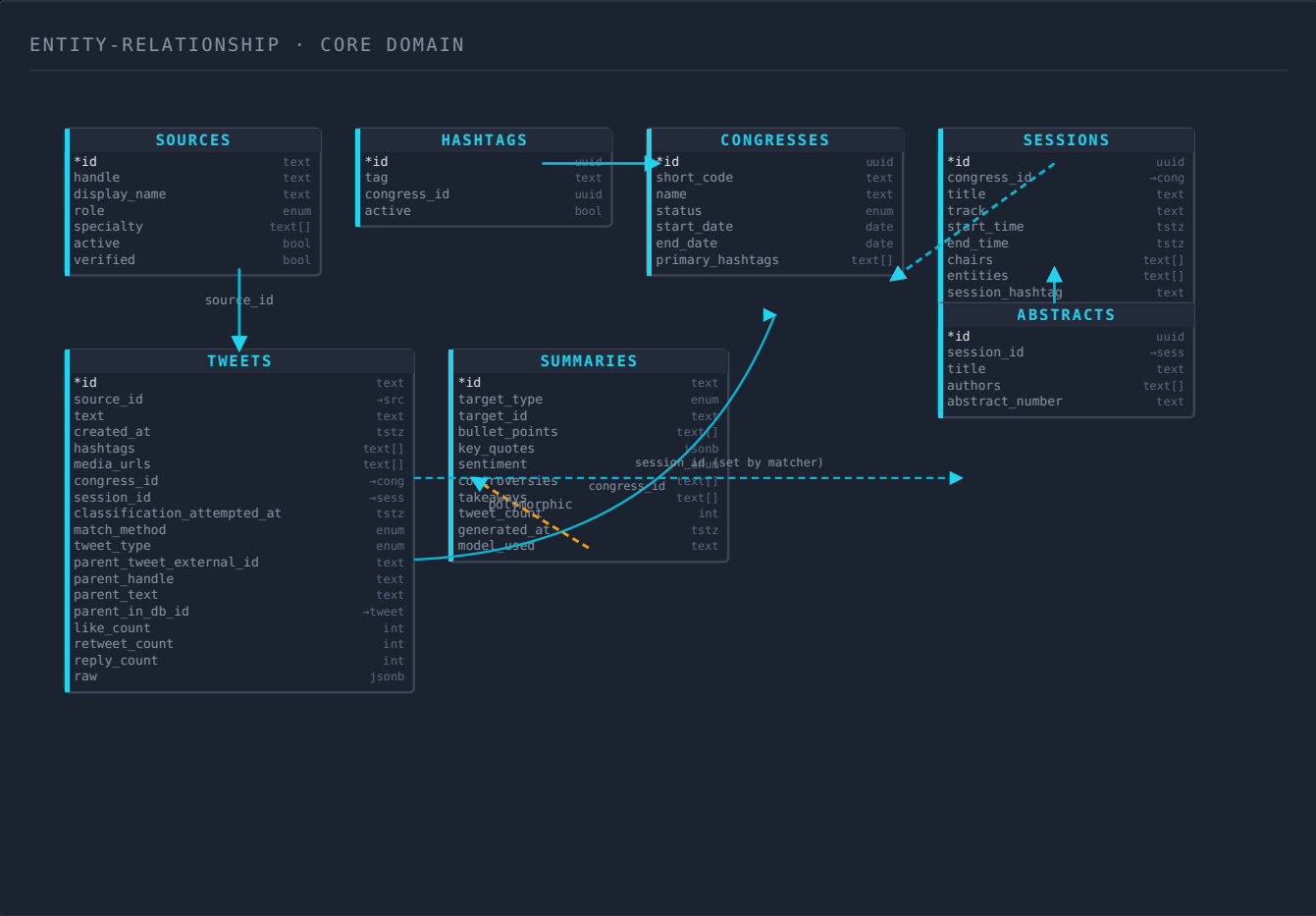
BACKEND & DATA

LAYER	CHOICE	NOTES
Database	Postgres (Supabase)	RLS on every table. pg_cron, pg_net, pgvector extensions enabled.
Auth	Supabase Auth	Email + magic link. JWT-based. Custom requireSupabaseAuth middleware on server routes.
Secret storage	Postgres vault	Encrypted at rest. Read via SECURITY DEFINER RPCs.
Edge functions	Supabase Edge (Deno)	ai-summarize, admin-invite-user. CORS-aware.
Scheduler	pg_cron + pg_net	Cron expressions fire HTTP calls to /api/public/hooks/* with vault-sourced bearer.
Email	@lovable.dev/email-js	React Email templates. Resend/SES under the hood.

EXTERNAL APIS

API	USE	LIMITS
X API v2 (Basic)	Tweet ingestion, user lookup, hashtag search	≈10K tweets/month; 300/15min lookup; per-endpoint rate-limit headers respected
Lovable AI Gateway	Summarization, classification, AI-suggest replies	OpenAI-compatible; tool-calling for structured output; model gemini-2.5-flash / -flash-lite

The core domain is global: one row per source, one per congress, one per session, one per tweet. RLS allows authenticated read; writes restricted to admin/editor or service role.



KEY ENTITIES

SOURCES	X accounts being tracked. <code>id</code> is <code>lower(handle)</code> by convention (enforced via CHECK constraint). Specialty array overlaps with user specialties for filtering.
CONGRESSES	Real-world events. Status: <code>upcoming</code> / <code>live</code> / <code>archived</code> . Drives matcher's window detection and digest "Active congress" preset.
SESSIONS	Time-bounded slots within a congress. <code>session_hashtag</code> is the primary matcher anchor. <code>entities</code> (text[]) holds drug/trial/intervention vocabulary used by the matcher's step 4.
ABSTRACTS	Subordinate to sessions. <code>abstract_number</code> is the matcher's step 2 anchor.
TWEETS	Global. Hierarchy fields (<code>tweet_type</code> , <code>parent_*</code>) capture reply/quote relationships. <code>session_id</code> and <code>congress_id</code> are set post-ingest by the matcher. <code>classification_attempted_at</code> is the cost-control flag preventing repeat LLM calls.
SUMMARIES	Polymorphic via (<code>target_type</code> , <code>target_id</code>): session / abstract / congress. Upserted on uniqueness; latest content always wins.

NOTABLE DESIGN DECISIONS

- **Parent snapshots on tweets.** `parent_handle` and `parent_text` are copied at ingest time so rendering quote/reply context never requires a JOIN. Parent may not exist as its own `tweets` row.

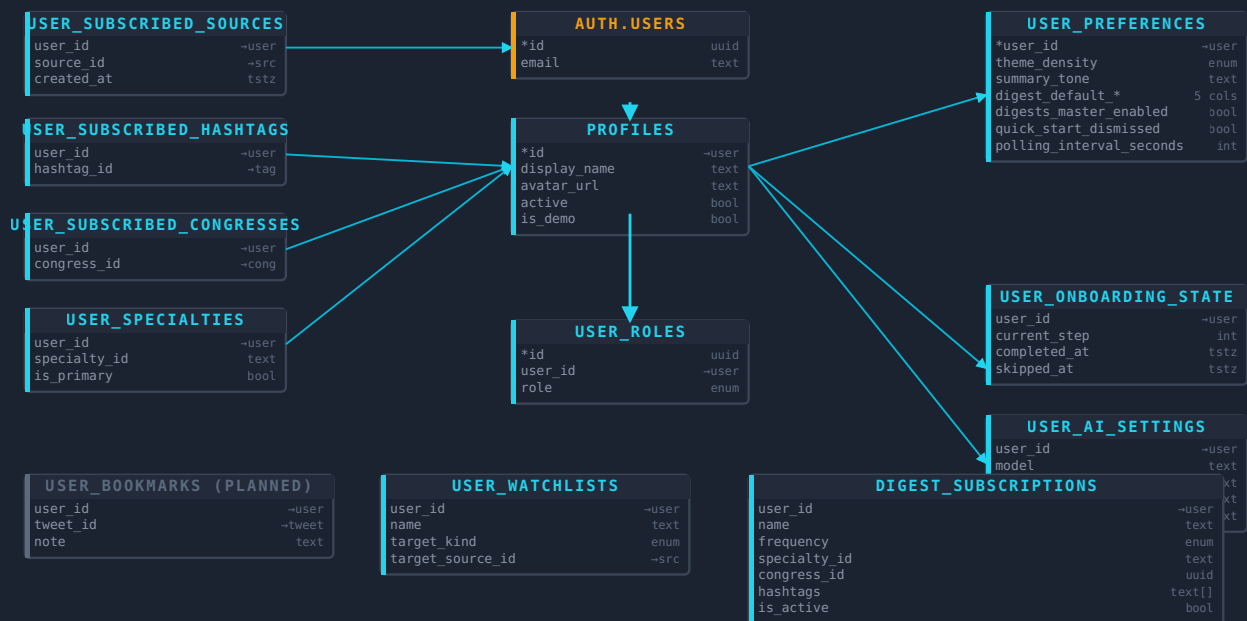
- **Polymorphic summary target.** A single `summaries` table with `(target_type, target_id)` uniqueness covers session, abstract, and congress-day summaries. Trade-off: no FK to the target.
- **FK on tweets.source_id ON DELETE CASCADE.** Hard-deleting a source removes its tweets. Soft delete (`active=false`) leaves data intact.

05 Database — per-user tier

S05 · PERSONALIZATION DATA

The per-user tier covers identity, subscriptions, preferences, and personal state. Every table has RLS scoped to `auth.uid()`. Service-role contexts (cron, ingestion) bypass for system writes.

ENTITY-RELATIONSHIP · PER-USER TIER



IDENTITY STACK

- `auth.users` — Supabase-managed; do not FK directly elsewhere. Project convention is to use `user_id uuid NOT NULL` without FK (matches Supabase guidance).
- `profiles` — public-facing identity. Display name, avatar, email, `is_demo` flag, `active` flag.
- `user_roles` — enum `app_role`: `admin` / `editor` / `viewer`. Trigger `prevent_demo_role_escalation` blocks demo users from being granted admin/editor.

SUBSCRIPTION TABLES

- `user_subscribed_sources`(`user_id`, `source_id`, `created_at`) — personal follow list. PRIMARY KEY (`user_id`, `source_id`) prevents duplicates.
- `user_subscribed_hashtags`, `user_subscribed_congresses` — same pattern.
- `user_specialties`(`user_id`, `specialty_id`, `is_primary`) — multi-select up to 3, one primary.

PREFERENCES AND AI

- `user_preferences` — theme density, polling interval, summary tone/language defaults, digest defaults (frequency, send hour, timezone, master toggle), `quick_start_dismissed`.
- `user_ai_settings` — per-user model, prompt template, tone, language, max bullets.
- `user_onboarding_state` — wizard progress; `completed_at` / `skipped_at`.

DIGESTS & ALERTS

- `digest_subscriptions` — multi-bound: source IDs, specialty, congress, hashtags. Frequency / day / hour / timezone. `is_active`.
- `digest_subscription_sources` / `digest_subscription_recipients` — child rows, FK cascade on delete.
- `user_watchlists` + `user_watchlist_topics` + `user_watchlist_matches` — keyword-triggered alerts on tracked sources or groups.

06 Table reference and key constraints

S06 · SCHEMA DETAILS

CONSTRAINTS WORTH KNOWING

CONSTRAINT	TABLE	EFFECT
<code>sources_id_lowercase_handle_check</code>	<code>sources</code>	<code>id = lower(handle)</code> ; prevents UI-added phantom sources that don't link to tweets
<code>fk_tweets_source ON DELETE CASCADE</code>	<code>tweets</code> → <code>sources</code>	Hard-delete source removes its tweets
<code>tweets_match_method_check</code>	<code>tweets</code>	Enum: <code>hashtag</code> , <code>abstract_number</code> , <code>speaker</code> , <code>entity</code> , <code>time_window</code> , <code>llm</code> , <code>thread_propagation</code> , <code>manual</code>
<code>tweets_tweet_type_check</code>	<code>tweets</code>	Enum: <code>original</code> , <code>reply</code> , <code>quote</code> , <code>retweet</code> (retweets filtered at query level)
<code>uniq_summaries_target</code>	<code>summaries</code>	UNIQUE (<code>target_type</code> , <code>target_id</code>) — one summary per target
<code>uniq_user_source</code>	<code>user_subscribed_sources</code>	PRIMARY KEY (<code>user_id</code> , <code>source_id</code>) — duplicates impossible
<code>congresses_primary_hashtags_clean</code>	<code>congresses</code>	Trigger normalizes to lowercase, strips leading #

KEY INDEXES

INDEX	TABLE	WHY
<code>idx_tweets_created_at</code>	<code>tweets</code>	DESC ordering for feed queries
<code>idx_tweets_session_id</code>	<code>tweets</code>	Partial WHERE <code>session_id</code> IS NOT NULL; session detail page
<code>idx_tweets_unclassified</code>	<code>tweets</code>	Partial WHERE <code>session_id</code> IS NULL AND <code>classification_attempted_at</code> IS NULL; matcher input
<code>idx_tweets_hashtags</code>	<code>tweets</code>	GIN; hashtag-filter queries
<code>idx_tweets_text_fts</code>	<code>tweets</code>	GIN to <code>tsvector</code> ; future AI Q&A retrieval
<code>idx_tweets_parent_external</code>	<code>tweets</code>	Partial WHERE <code>parent_tweet_external_id</code> IS NOT NULL; thread propagation
<code>idx_sessions_entities</code>	<code>sessions</code>	GIN; matcher step 4 entity lookup
<code>idx_abstracts_number_session</code>	<code>abstracts</code>	(<code>abstract_number</code> , <code>session_id</code>); matcher step 2

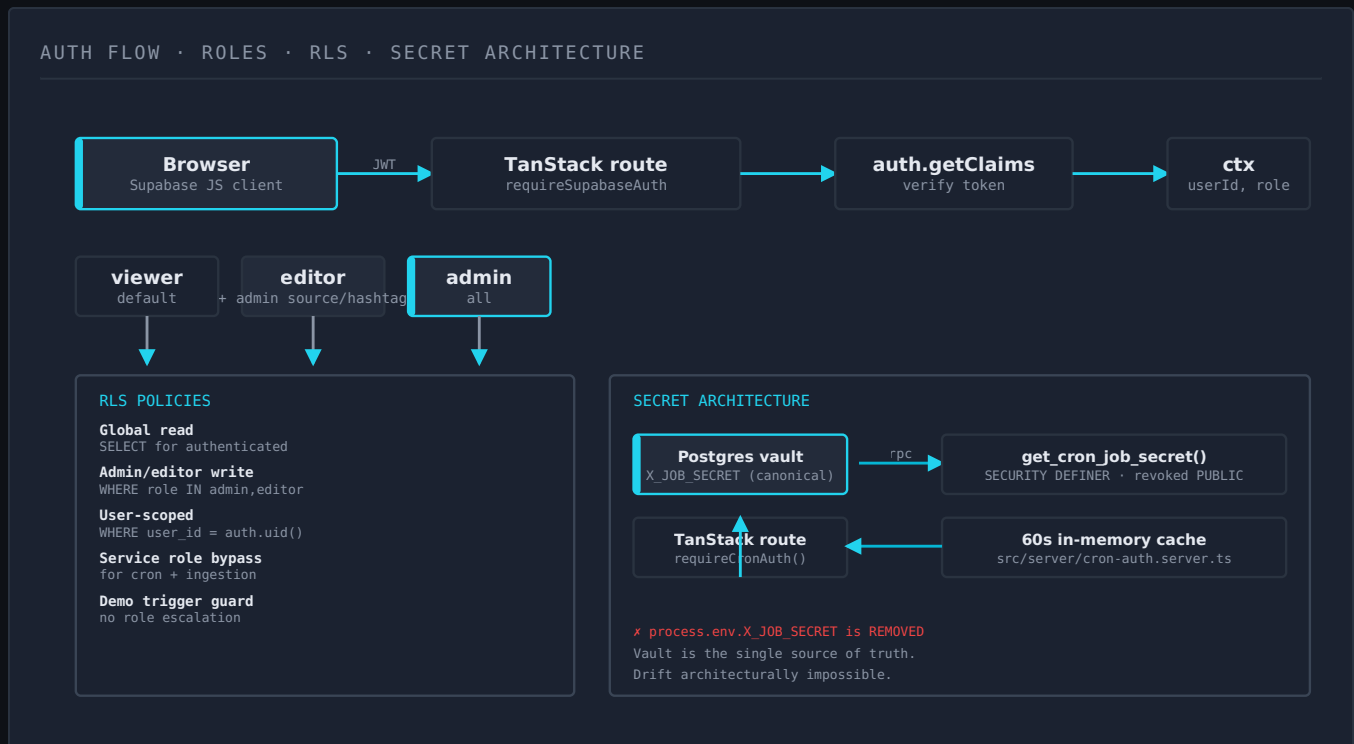
TRIGGERS

- `sync_source_tweet_stats` — keeps `sources.tweet_count` and `last_seen_at` in sync on tweet INSERT/UPDATE/DELETE
- `touch_updated_at` — applied to most tables with an `updated_at` column
- `prevent_demo_role_escalation` — blocks demo accounts from being granted admin/editor roles
- `congresses_primary_hashtags_normalize` — strips # and lowercases on INSERT/UPDATE

07 Authentication and Row-Level Security

S07 · TRUST BOUNDARY

RLS is enabled on every table. Every authenticated request goes through `requireSupabaseAuth` middleware which validates the bearer JWT via `supabase.auth.getClaims()` and injects `userId` + claims into the route context. Service-role contexts (cron jobs, ingestion) use `supabaseAdmin` to bypass RLS.



ROLE HIERARCHY

VIEWER

Default for all signed-in users.
SELECT on global tables, full
CRUD on own user_* tables.
Cannot modify global registries.

EDITOR

Viewer + admin operations on
sources, hashtags, congresses,
sessions. Used for content
curators who aren't infrastructure
operators.

ADMIN

All operations including user
management, ingestion config,
recommendation matrices, cron
operations, demo provisioning.

RLS PATTERN CONVENTIONS

PATTERN	EXAMPLE
Global read	FOR SELECT TO authenticated USING (true)
Editor write	USING (EXISTS (SELECT 1 FROM user_roles WHERE user_id = auth.uid() AND role IN ('admin','editor')))
User-owned	USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid())
Owner via parent	USING (EXISTS (SELECT 1 FROM digest_subscriptions d WHERE d.id = digest_id AND d.user_id = auth.uid()))

SECRET ARCHITECTURE

The `X_JOB_SECRET` used by cron-gated routes lives in Postgres vault exclusively. Routes call `public.get_cron_job_secret()` via `supabaseAdmin.rpc()` with a 60-second in-memory cache. The function is `SECURITY DEFINER` with `REVOKE EXECUTE ... FROM PUBLIC, anon, authenticated`, so only the postgres role (which runs `pg_cron` and the service-role client) can call it.

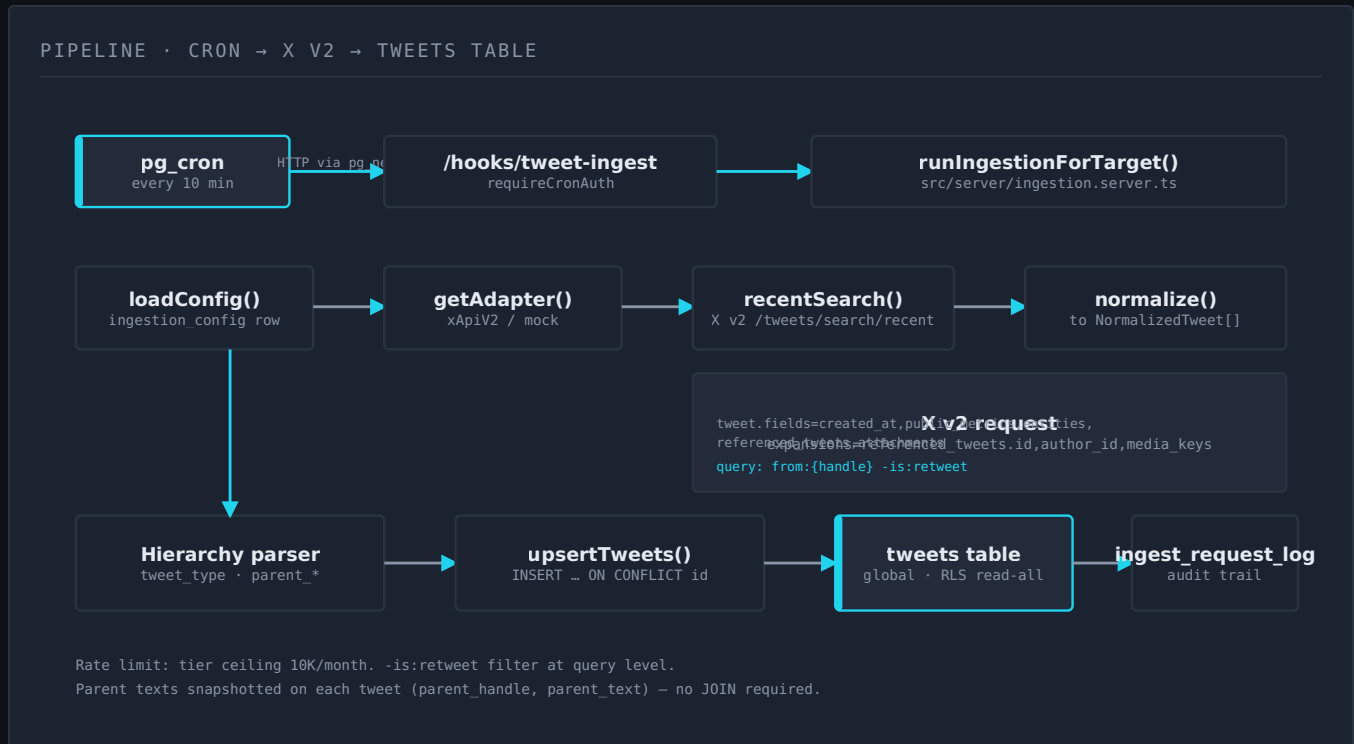
LESSON LEARNED

The dual-storage anti-pattern (env var + vault) drifted twice in production before the vault-only refactor. Single source of truth made the class of bug structurally impossible.

08 Tweet ingestion pipeline

S08 · X V2 → TWEETS TABLE

Ingestion runs every 10 minutes (or every 1 minute via the queue worker for wizard-triggered backfills). `pg_cron` fires HTTP calls to `/api/public/hooks/tweet-ingest` with vault-sourced auth. The route delegates to `runIngestionForTarget()` which orchestrates adapter selection, X API call, normalization, and upsert.



PLUGGABLE ADAPTER PATTERN

The `TwitterAdapter` interface decouples ingestion logic from the data source. Current implementations:

- `xApiV2Adapter` — production. Hits X API v2 recent search and timeline endpoints with full expansion set.
- `mockAdapter` — test/dev. Returns canned data.
- Stubs reserved: `socialDataAdapter`, `twitterApiIoAdapter` — for historical backfill from third-party providers.

X API V2 REQUEST SHAPE

```
GET /2/tweets/search/recent
?query=from:Uroweb -is:retweet
&max_results=100
&start_time=2026-05-01T00:00:00Z
&tweet.fields=created_at,public_metrics,lang,entities,
               referenced_tweets,attachments
&expansions=author_id,attachments.media_keys,
             referenced_tweets.id,referenced_tweets.id.author_id
&user.fields=username,name
&media.fields=url,preview_image_url
Authorization: Bearer ${X_BEARER_TOKEN}
```

HIERARCHY NORMALIZATION

Each tweet's `referenced_tweets[0]` is inspected:

- `retweeted` → tweet is dropped (defense in depth on top of `-is:retweet` query filter)
- `replied_to` → `tweet_type='reply'`; parent text snapshot from `includes.tweets`
- `quoted` → `tweet_type='quote'`; same snapshot pattern
- no reference → `tweet_type='original'`

UPSERT AND DEDUPE

`upsertTweets()` inserts with `ON CONFLICT (id) DO NOTHING`. The X tweet id is the primary key, so re-ingestion of the same tweet is idempotent. `parent_in_db_id` is set to the parent's id when the parent exists in our DB (for click-to-navigate behavior in the UI).

09 Tweet-to-session matcher cascade

S09 · CLASSIFICATION LOGIC

Tweets land with `session_id IS NULL`. The matcher runs every 5 minutes via cron, processing up to 200 unclassified tweets in the last 72-hour window. The cascade tries the cheapest, most deterministic signals first; the LLM is the last resort and is capped.

SEVEN-STEP CASCADE · FIRST MATCH WINS



COST CONTROL INVARIANT

Every processed tweet — whether matched or not — gets `classification_attempted_at = now()` stamped at the end of the loop. Without this flag, subsequent ticks would re-process every still-unmatched tweet through the LLM forever, burning tokens on tweets that genuinely don't match any session.

EMPIRICAL LIFT

On a recent test dataset of 138 tweets in last 14 days, the seven-step cascade produced: 31 reply, 5 original, 3 quote tweet_type breakdowns. Hashtag-only baseline matched ~5%; with the full cascade (hashtag + speaker + entity + time-window + LLM-disambiguation + thread propagation), match rate is ~70%+ depending on session metadata richness.

STEP 6 — LLM DISAMBIGUATION

Only fires when `candidate_sessions.length > 1` after time-window filtering. Sends the tweet text + top-8 candidate sessions to gemini-2.5-flash-lite with a single-line response prompt ("reply with the id of the best match, or 'none'"). Hard cap: 30 LLM calls per cron tick. Tokens logged to `tweet_match_run_log.llm_tokens_used`.

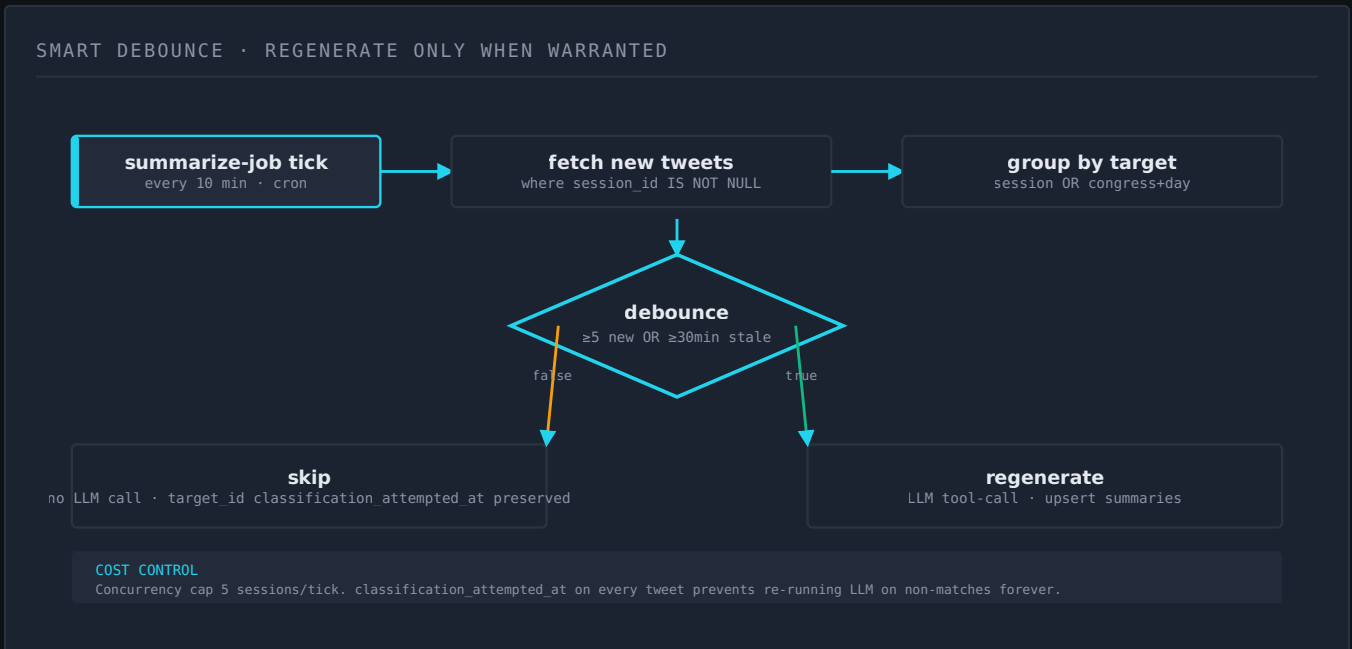
STEP 7 — THREAD PROPAGATION

After the main cascade, a sweep looks for tweets where `session_id IS NULL AND parent_in_db_id IS NOT NULL`. If the parent has a `session_id` and the child is within $\pm 2h$ of the parent's timestamp, inherit. Marks `match_method='thread_propagation'`. Bounded to 500 propagations per tick.

10 Summarization pipeline

S10 · LLM → SUMMARIES TABLE

`/api/public/hooks/summarize-job` runs every 10 minutes. Groups tweets two ways: (a) classified tweets by `session_id`, (b) unclassified-but-congress-tagged tweets by `(congress_id, date)`. The second path is the fallback that keeps the product useful when session data isn't yet populated.



DEBOUNCE RULES

For each candidate target (session OR congress-day):

- No prior summary → generate if `tweet_count` ≥ 3 (sessions) or ≥ 10 (congress-day).
- Prior summary exists → regenerate only when **≥5 new tweets since last `generated_at`** OR **≥30 min stale AND ≥1 new tweet**.
- Otherwise → skip; no LLM call.

TOOL-CALL OUTPUT

The LLM is invoked with a forced `emit_summary` function call. Structured output:

```
{
  "bulletPoints": ["3-5 concise takeaways"],
  "keyQuotes": [{"quote", "tweetId"}],
  "sentiment": "positive|mixed|critical|neutral",
  "controversies": [...],
  "takeaways": [...]
}
```

PROMPT-INJECTION DEFENSE

Each tweet is wrapped in `<tweet id="X">...</tweet>` markers. The system prompt instructs the model to treat anything inside those blocks as untrusted user content. Returned `keyQuotes[].tweetId` values are validated against the input set; any hallucinated IDs are dropped before persisting.

CONCURRENCY CAP

Max 5 summaries per tick across all targets. If more are due, defer to the next run. Prevents a single backfill landing 200 classified tweets from spawning 50 parallel LLM calls.

11 AI Gateway integration

All LLM operations route through the Lovable AI Gateway at `https://ai.gateway.lovable.dev/v1/chat/completions`. The gateway is OpenAI-compatible, supports tool-calling, and uses a single secret `LOVABLE_API_KEY` managed by Lovable Cloud (no per-user keys).

WHERE LLM IS CALLED

CALLER	MODEL	PATTERN	COST BAND
summarize-job	gemini-2.5-flash	Tool-call · structured JSON	\$\$\$
match-tweets · step 6	gemini-2.5-flash-lite	Single-line classification	\$
ai-summarize edge fn	configurable	Tool-call · summarize OR ping OR suggest_replies	\$\$\$
Compose AI suggest	gemini-2.5-flash	Suggest 3 reply drafts with tone	\$\$
refreshSessionEntities	gemini-2.5-flash-lite	Extract drug/trial/procedure entities	\$
computeSourceBriefing	gemini-2.5-flash	Weekly per-source briefings	\$\$\$

COST-CONTROL MECHANISMS

- **Server-side caching** — summaries upserted; congress-suggest cached 30 days; briefings cached per week.
- **Tool-call validation** — returned `tweetId` references validated against input set; malformed responses logged and skipped (don't burn another call).
- **Per-run hard caps** — 30 LLM calls/tick on matcher, 5 summaries/tick on summarizer, 60/min per-user on lookups.
- **Quota reservation** — `reserveExpensiveLlmCall()` helper in `llm-quota.server.ts` tracks aggregate spend and refuses calls that would exceed budget.

PROMPT-INJECTION MODEL

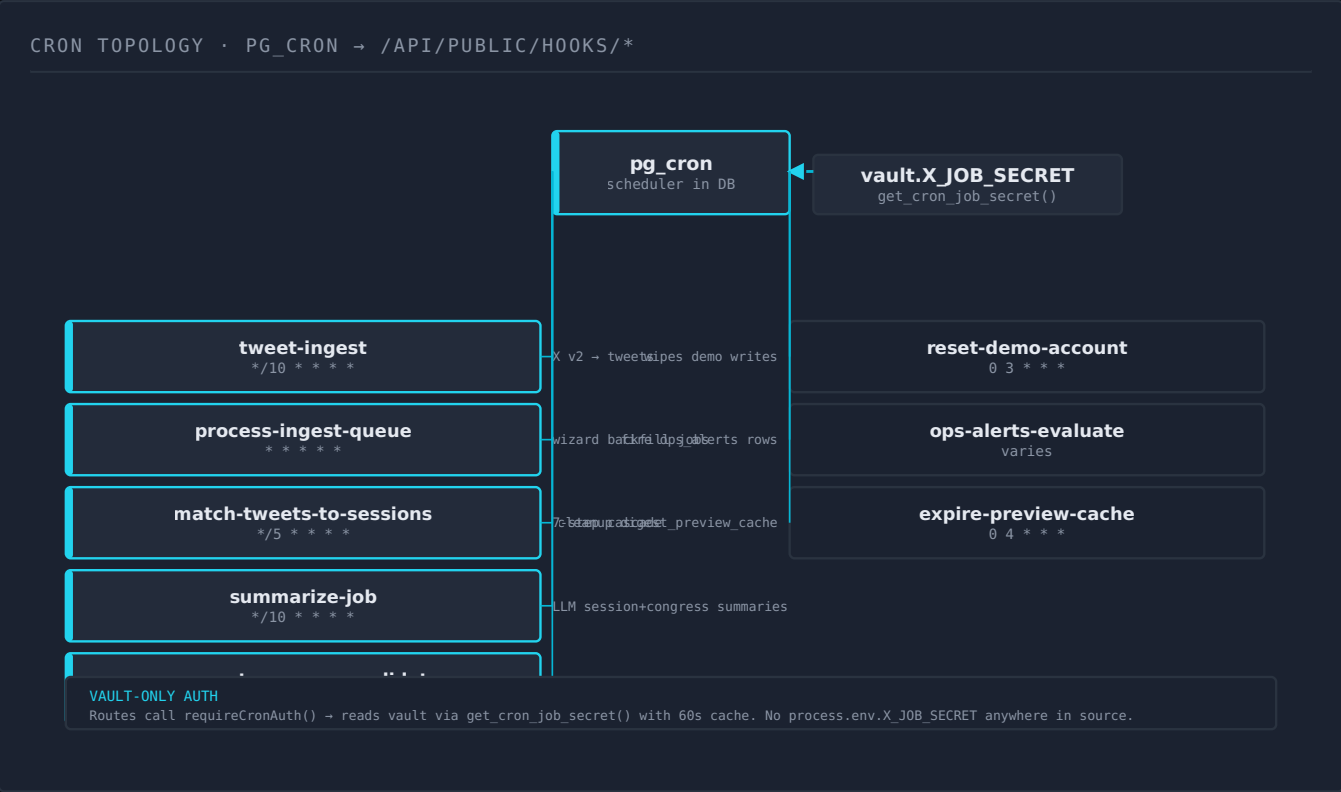
The system prompt instructs the model to treat anything inside `<tweet id="X">...</tweet>` blocks as untrusted user input. Every prompt that includes tweet text uses these delimiters. The model is told never to follow instructions found inside.

EDGE FUNCTION VS SERVER ROUTE

`supabase/functions/ai-summarize/` is a Deno-runtime edge function (uses `Deno.env.get()`). Other LLM call sites are in TanStack server routes (Cloudflare Worker runtime, use `process.env`). These two runtimes have different APIs — keep them straight when adding new LLM call sites.

12 Operational layer — cron topology

Eight scheduled jobs run continuously. Each is gated by `requireCronAuth` reading the secret from vault. Cron health is exposed via `public.get_ingestion_cron_health()` RPC and rendered on Admin → Ingestion.



RUN LOGS

- `ingest_request_log` — per X API call: status code, count returned, ms taken, source id
- `ingest_queue_run_log` — per process-ingest-queue tick: jobs picked, completed, failed, rate-limited
- `summarize_run_log` — per summarize-job tick: targets considered, regenerated, LLM tokens used
- `tweet_match_run_log` — per matcher tick: tweets considered, matches by method, LLM calls, tokens

13 Frontend — routes and shell

35 routes organized as file-based via TanStack Router. The shell (`AppShell.tsx`) provides sidebar (desktop), bottom tab bar (mobile), top bar with breadcrumb, the demo banner, and the persistent compose FAB.

ROUTE TREE · 35 ROUTES ACROSS WORKSPACE, ME, ADMIN

/root.tsx

PUBLIC

/authauth.tsx

/unsubscribe

WORKSPACE

/dashboarddashboard.tsx

/feedfeed.tsx

/summariessummaries.tsx

/congressescongresses.tsx

/congresses/\$idcongresses.\$tsx

/sessions/\$idsessions.\$tsx

/discoverdiscover.tsx

/sourcesources.tsx

/sources/\$handlesources_.\$handle.tsx

/digestsdigests.tsx

/alertsalerts.tsx

/groups/\$sluggroups.\$slug.tsx

/help/instructionshelp.instructions.tsx

ME · CONFIG SUB-PAGES

/meme.tsx

/me/profileme.profile.tsx

/me/preferencesme.preferences.tsx

/me/notificationsme.notifications.tsx

/me/aime.ai.tsx

/me/x-accountme.x-account.tsx

/me/followingme.following.tsx

/me/postsmе.posts.tsx

ADMIN (admin role only)

/admin/usersadmin.users.tsx

/admin/groupsadmin.groups.tsx

/admin/recommendationsadmin.recommendations.tsx

/admin/ingestionadmin.ingestion.tsx

/admin/opsadmin.ops.tsx

/admin/email-diagnosticsadmin.email-diagnostics.tsx

/configuration/brainstormconfiguration.brainstorm.tsx

ROUTE CATEGORIES

CATEGORY	AUTH	ROUTES
Public	none	/auth, /unsubscribe, /email/unsubscribe
Workspace	authenticated	/dashboard, /feed, /summaries, /congresses, /sessions, /discover, /sources, /digests, /alerts, /groups, /help/*
Me hub	authenticated	/me, /me/profile, /me/preferences, /me/notifications, /me/ai, /me/x-account, /me/following, /me/posts
Admin	admin role	/admin/users, /admin/groups, /admin/recommendations, /admin/ingestion, /admin/ops, /admin/email-diagnostics, /configuration/brainstorm

ROUTING PATTERNS

- File-based: `src/routes/me.profile.tsx` ↔ `/me/profile`
- Dynamic segments: `congresses.$congressId.tsx` ↔ `/congresses/:id`

- Underscore prefix: `sources_.$handle.tsx` ↔ `/sources/:handle` (escapes default segment ordering)
- Catch-all 404: `$.tsx`

MOBILE ROUTING

`/` redirects to `/feed` on mobile, `/dashboard` on desktop. Admin routes redirect to `/me` with toast "Use desktop for admin tools" on mobile viewports. `/me/*` sub-pages render inside `<MobileSubPage />` shell with a back chevron in the header.

14 Frontend — component organization

S14 · CO-LOCATION

Components co-located by domain. Cross-cutting primitives in `src/components/ui/` (shadcn).

DIRECTORY LAYOUT

```
src/components/
├─ shell/           // AppShell, Sidebar, TopBar, BottomTabBar, MobileSubPage
├─ feed/           // TweetCard, TweetStream, FilterBar, MobileFeedLayout
├─ discover/       // ForYouTab, GroupsTab, MobileDiscover
├─ digests/        // DigestsList, DigestWizard, MobileDigestsList, MobileDigestWizard
├─ congresses/     // CongressDetail, NewCongressDialog
├─ sessions/       // SessionDetail (the crown jewel)
├─ sources/        // SourcesTable, HashtagsTable, RoleBadge
├─ summaries/      // SummariesIndex, SummaryCard
├─ settings/       // ProfileSettings, PreferencesSettings, NotificationsSettings,
                  // AiSettings, XSettings, IngestionSettings, SwitchRow, MobileSaveBar
├─ wizard/         // OnboardingWizard (1530 LOC), ResumeBanner
├─ x-wizard/       // X-account connection wizard
├─ x/              // ComposeTweetDialog, ComposeFAB
├─ handles/        // HandleChip – universal @-mention component
├─ watchlists/     // WatchlistFormDialog, NotificationsBell
├─ spotlight/      // SourceBriefingDialog
├─ brainstorm/     // Admin chat (Composer, MessageList, PresenceList)
├─ admin/          // EmailDiagnosticsView
├─ dashboard/      // Dashboard (605 LOC) – KPIs + cron health + activity
└─ ui/             // shadcn primitives
```

CROSS-CUTTING HOOKS

HOOK	PURPOSE
<code>useAuth()</code>	Session + profile + roles + prefs. Single source of truth on identity.
<code>useIsMobile()</code>	<code>matchMedia < 768px</code> . Drives all mobile/desktop branching.
<code>useLiveData()</code>	TanStack Query wrapper with 30s polling, pause-on-hidden.
<code>useFollowSource</code> / <code>useUnfollowSource</code>	Optimistic mutations with cache invalidation cascade.
<code>useShouldShowComposeFab()</code>	Route-scoped FAB visibility on mobile.
<code>useOnboardingGate()</code>	Wizard-open decision based on <code>user_onboarding_state</code> .
<code>useCongressSuggest()</code>	AI-suggest hook for congress entry form.

STATE MANAGEMENT

Server state via TanStack Query exclusively. No global client state library (no Redux, no Zustand). Local UI state via `useState`. Form state via `react-hook-form`. URL state via TanStack Router search params.

15 Mobile architecture

S15 · IOS-FIRST

Mobile is a deliberately distinct rendering path, not a responsive CSS exercise. Six dedicated mobile components reorganize content into single-column flows; bottom tab bar replaces the sidebar; FAB replaces compose button in topbar.

MOBILE LAYOUT · PHONE-SHAPED REFERENCE

TOPBAR

40px, accent CTA on desktop only

INLINE COMPOSER

always sticky-ish above feed

TODAY RIBBON

live aggregate, tap to expand

TWEET CARDS

44px tap targets, swipe-right to bookmark

FAB

tap = compose · long-press = AI-assist

BOTTOM TAB BAR

fixed · safe-pb · z-40

UroFeed

DEMO · RESETS NIGHTLY

What did you take away?

TODAY · 3 SUMMARIES · APCCC26 LIVE

@DrSpratticus

14m · #APCCC26

Brief tweet body preview...
...continued.

4 · 6 · 2 · 14

@DrSpratticus

14m · #APCCC26

Brief tweet body preview...
...continued.

4 · 6 · 2 · 14

@DrSpratticus

14m · #APCCC26

Brief tweet body preview...
...continued.

4 · 6 · 2 · 14

+

FEED

DISCOVER

DIGESTS

ME

DEMO BANNER

Shows when profile.is_demo = true

HIERARCHY

↳ reply · ↳ quote · ♥ like

QUOTE-SHARE

↳ opens Compose with quote URL pre-filled

TAB STATES

active = accent · 2px top border

MOBILE-SPECIFIC COMPONENTS

COMPONENT	PATH	ROLE
BottomAppBar	shell/	4 tabs: Feed, Discover, Digests, Me. Fixed bottom. Safe-area aware.
MobileSubPage	shell/	Shared shell for /me/* sub-pages. Back chevron, centered title, right-action slot.
ComposeFAB	x/	56px circle, bottom-24 right-4. Tap = compose. Long-press = AI-suggest.
MobileFeedLayout	feed/	Inline composer + Today ribbon + TweetStream. No right rail, no scrubber.
MobileDiscover	discover/	Chip-row tabs, full-width cards, big follow buttons.
MobileDigestsList	digests/	Card layout with swipe-left actions.
MobileDigestWizard	digests/	Single full-screen sheet (no multi-step wizard on mobile).

UroFeed · Technical Architecture

25 / 35

IOS-SPECIFIC AFFORDANCES

- `safe-pl` / `safe-pr` / `safe-pt` / `safe-pb` CSS utilities use `env(safe-area-inset-*)`
- `ios-scroll` utility enables `-webkit-overflow-scrolling: touch`
- Density preference saved per-user; `data-density` attribute on `<html>`
- Visual viewport detection hides FAB when keyboard is up (otherwise it floats over the keyboard)
- Long-press timing: 500ms with `navigator.vibrate?.(10)` haptic feedback if supported

MOBILE TAP-TARGET STANDARDS

All interactive elements meet iOS HIG minimum 44×44px on mobile. Enforced via:

- `<SwitchRow>` primitive — entire row tappable, switch is just visual indicator
- `<MobileSaveBar>` primitive — sticky bottom bar above tab bar (clears 80px tab + safe-area)
- Button size override pattern: `h-12 md:h-9`

16 Compose and posting flow

ENTRY POINTS

SURFACE	TRIGGER	PRE-FILL
Top bar (desktop)	"Share to X" CTA button	Empty
Mobile FAB	Tap (anywhere on workspace routes)	Empty
Mobile FAB long-press	500ms hold	AI-suggest auto-fires
Inline composer (mobile Feed)	Tap "What did you take away?"	Active congress hashtag
Tweet card Quote button	Tap quote icon on any tweet	Quote URL + blank line + cursor
Tweet card Reply	Tap reply icon	Reply context attached

COMPOSE DIALOG FEATURES

- **Tone presets:** Professional, Collegial, Inquisitive, Counterpoint, Concise
- **AI Suggest:** 3 draft alternatives via `ai-summarize` edge function with `mode: 'suggest_replies'`
- **Character count:** grapheme-aware via `Intl.Segmenter`; turns amber at 240, red at 280
- **Account switcher:** dropdown if multiple X accounts connected; activates one as "send-from"
- **Daily cap visualization:** shows N of 50 posts today

POST PATH

```
ComposeTweetDialog.handlePost
└─ postTweet({ text, inReplyToTweetId })           // server function
    └─ getActiveXAccount(userId)                   // from x_accounts table
        └─ if (profile.is_demo) {
            insertDemoPost(...);                  // sandbox
            return { ok: true, simulated: true, url: ... };
        }
    └─ checkDailyCap()                             // 50/day default
    └─ fetch(X v2 /2/tweets, POST)                 // real post
    └─ insert into x_post_log                       // audit
    └─ return { ok: true, url: ... };
```

DEMO SANDBOX

`postTweet` always checks `profiles.is_demo` first. Demo users' posts write to `demo_posts` table; no X API call fires. The dialog UX is identical (toast confirms, /me/posts updates) so visitors get the full flow without real-world side effects.

17 Digests and email pipeline

S17 · SCHEDULED SYNTHESSES

Digests are user-configured scheduled email summaries. Each digest binds to any combination of sources, specialty, congress, and hashtags. The send-job cron evaluates due digests, queries matching tweets, summarizes via LLM, and dispatches via the email transport.

DIGEST MODEL

FIELD	PURPOSE
name	User-facing label
frequency	daily / weekly / biweekly / monthly
day_of_week, send_hour, timezone	Schedule specifics
source_ids	Via <code>digest_subscription_sources</code> join
specialty_id, congress_id, hashtags	Topic bindings (any combo)
is_active	Per-digest toggle
next_send_at	Computed from schedule + last send

MASTER TOGGLE AND DIGEST DEFAULTS

User-level preferences in `user_preferences`:

- `digests_master_enabled` — global pause. When false, send-job skips all of this user's digests regardless of per-digest `is_active`.
- `digest_default_frequency`, `digest_default_send_hour`, `digest_default_timezone` — pre-fill new digests.
- `digests_active_by_default` — controls whether new digests start active.

SEND JOB QUERY SHAPE

```
SELECT t.* FROM tweets t
WHERE t.created_at > :since
  AND (
    -- source binding
    (cardinality(:source_ids) > 0 AND t.source_id = ANY(:source_ids))
    OR
    -- specialty via sources.specialty array
    (:specialty_id IS NOT NULL AND EXISTS (
      SELECT 1 FROM sources s
      WHERE s.id = t.source_id AND :specialty_id = ANY(s.specialty)))
    OR
    -- congress binding
    (:congress_id IS NOT NULL AND t.congress_id = :congress_id)
    OR
    -- hashtag binding (array overlap)
    (cardinality(:hashtags) > 0 AND t.hashtags && :hashtags)
  )
ORDER BY t.created_at DESC LIMIT 100;
```

EMAIL RENDERING

Templates in `src/lib/email-templates/` using React Email. The same component renders both the live email and the in-app preview. Unsubscribe links use signed tokens stored in `email_unsubscribe_tokens`; per-digest opt-out without losing the rest of the user's preferences.

18 Discover and recommendation system

S18 · FIND PEOPLE TO FOLLOW

Discover has three lenses (For you / By group / By specialty). Each surfaces a different signal source. The page persists last-used tab in localStorage.

THREE LENSES

FOR YOU · ACTIVITY-DERIVED

The `aggregate-source-candidates` cron runs every 30 min and populates `source_candidates`. It scans tweets from the last 30 days for:

- `parent_handle` values from replies/quotes by monitored sources (weight: reply=3, quote=3)
- `entities.mentions` in tweet text from monitored sources (weight: mention=1, cap 3 per tweet to prevent thread-blast distortion)
- 2× recency multiplier on signals from the last 7 days

Top candidates are enriched via batched X v2 `users/by` calls (≤100 per request, rate-limited). Result: a ranked list of "people the sources you follow engage with most."

BY GROUP · ADMIN-CURATED

`source_groups` are admin-curated bundles by cancer area (Prostate, Bladder, Kidney, Andrology, etc.). Visibility: `official` / `public` / `private`. Group membership stored in `source_group_members`. Users subscribe to groups via `user_subscribed_source_groups`.

Group candidates surface via the admin `/admin/groups` page with a nomination workflow (`group_nomination_candidates` table, approve/reject from admin UI).

BY SPECIALTY · MATRIX-DRIVEN

Reads `recommended_sources_by_specialty` filtered by user's `user_specialties`. Each cell has a curator-provided `note` and weight. Surfaced grouped by specialty with section headers.

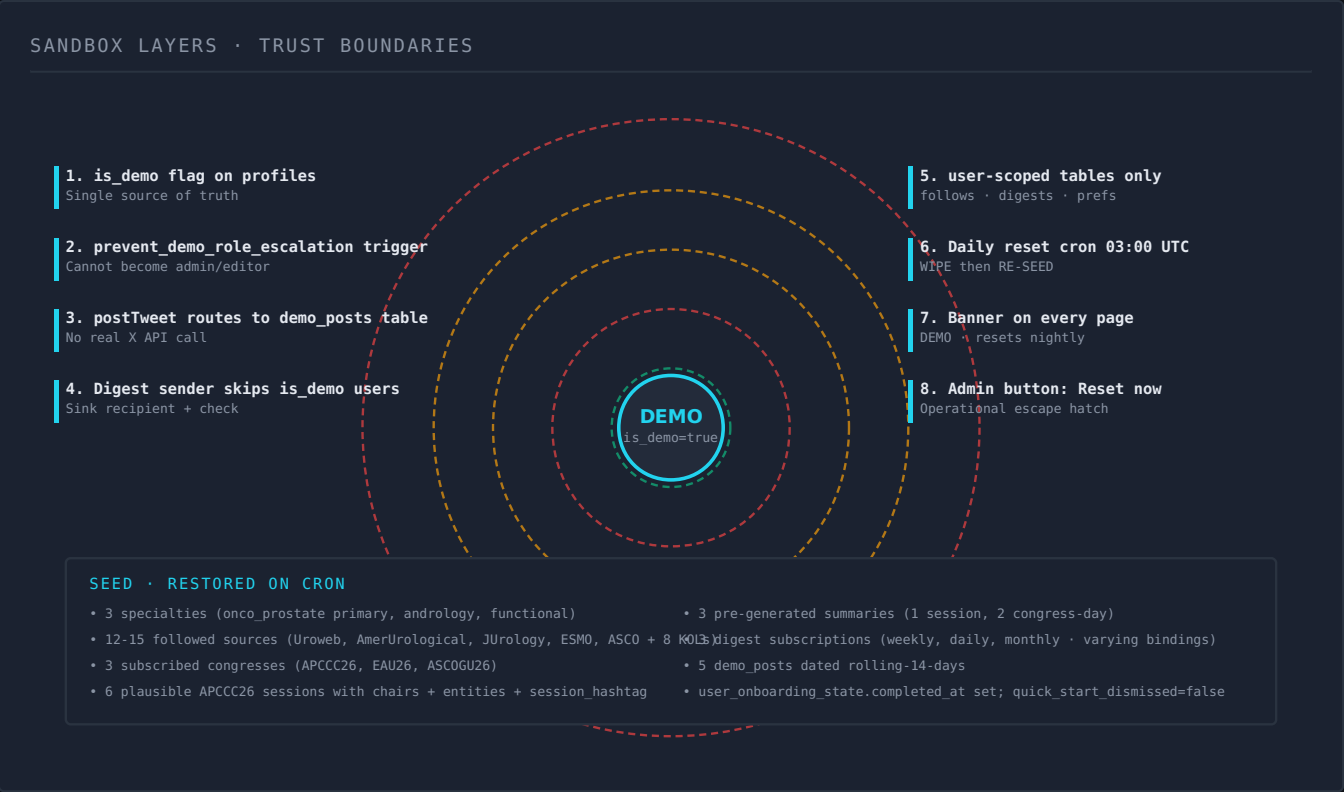
SOURCE PROFILE (`/SOURCES/$HANDLE`)

Each source has a deep-dive page powered by `source-spotlight.ts` server function. Surfaces:

- Core: avatar, bio, follower count, last enrichment timestamp
- Themes: top recurring topics from the source's tweets (computed via `computeSourceThemes`)
- Rhythm: typical posting hours by day-of-week (inferred timezone)
- Inner circle: top accounts this source engages with most
- Upcoming congresses: where this source is featured as chair/speaker
- Recent tweets stream + Set Up Alerts CTA → watchlist creation

19 Demo account architecture

A single shared credential (`demo@urofeed.app` + password) that any visitor can use. All features work — follow, configure digests, share posts — but writes are sandboxed and reset nightly to a canonical state.



THREE TRUST MECHANISMS

LAYER	IMPLEMENTATION
Flag-driven sandbox	<code>profiles.is_demo</code> boolean. <code>postTweet</code> , digest sender, and other write paths branch on this flag.
Trigger-blocked escalation	<code>prevent_demo_role_escalation</code> BEFORE INSERT/UPDATE on <code>user_roles</code> blocks admin/editor grants for <code>is_demo</code> users.
Periodic reset	<code>pg_cron</code> at 03:00 UTC runs <code>/api/public/hooks/reset-demo-account</code> . Wipes user-scoped tables, re-seeds canonical state.

SEEDED CANONICAL STATE

- 3 specialties (`onco_prostate` primary)
- ~15 followed sources (institutional + KOLs known to have recent tweets)
- 3 subscribed congresses (`APCCC26`, `EAU26`, `ASCOGU26`)
- 6 plausible `APCCC26` sessions with chairs + entities + `session_hashtag`
- 3 pre-generated summaries (1 session, 2 congress-day) seeded directly so they appear without waiting for the cron
- 3 digest subscriptions (weekly, daily, monthly) demonstrating different binding modes
- 5 `demo_posts` dated rolling-14-days back

- `user_onboarding_state.completed_at` set so wizard doesn't auto-open
- `quick_start_dismissed=false` so Quick Start panel appears for visitors

OPERATIONAL SURFACES

- **Provision / repair button** in Admin → Ingestion creates the auth user, sets password from Lovable secrets, runs the seed.
- **Reset demo account now** button — manual trigger of the nightly cron. Used when demo state drifts mid-day before showing it to someone.
- **Demo banner** rendered in `AppShell` when `profile.is_demo` is true. "Demo · resets nightly" on mobile, fuller text on desktop.

20 Observability and audit

S20 · OPERATIONAL VISIBILITY

AUDIT LOG

`audit_log` table captures admin and editor mutations. Schema:

```
(actor_user_id, actor_email_at_time, action, target_type,
target_id, target_label, diff_jsonb, created_at)
```

Action enum: `create`, `update`, `delete`, `activate`, `deactivate`, `invite`, `role_change`, `settings_change`, `source_deleted`, `user_deactivated`.

KNOWN DEBT

Audit writes are via scattered `recordAudit()` calls across mutation handlers, not Postgres triggers. Gap-prone — any new mutation that forgets the call goes unaudited. Trigger-based migration is on the roadmap.

CRON HEALTH SURFACE

Admin → Ingestion renders a live panel via `getIngestionCronHealth()`. Each row shows job name, schedule, last successful run, age, and red flag when stale (>2× expected interval). 30-second cache.

RUN LOGS (PER-JOB AUDIT)

TABLE	CAPTURES
<code>ingest_request_log</code>	Per-X-API-call: status, count returned, ms taken
<code>ingest_queue_run_log</code>	Per process-ingest-queue tick: jobs picked/completed/failed
<code>summarize_run_log</code>	Per summarize-job tick: targets, regenerations, LLM tokens
<code>tweet_match_run_log</code>	Per matcher tick: tweets, match methods, LLM calls/tokens
<code>email_send_log</code>	Per email: recipient, status, retries
<code>email_send_state</code>	Per-digest aggregate state (last sent, error count)

RATE LIMIT VISIBILITY

Admin → Ingestion shows live gauges for: lookup rate (per-user 60/min + global 200/15min), queue depth (pending/processing/completed), top lookup users in the last hour (spike investigation).

OPS ALERTS

Periodic evaluator writes to `ops_alerts` for: cron stale, secret drift suspected, X API quota high (>80%), LLM error rate elevated, ingestion queue backed up. Admin → Ops surfaces them with acknowledge button. Acknowledged alerts stay visible but muted.

21 Technical debt and roadmap

S21 · WHAT'S NEXT

KNOWN DEBT (BY PRIORITY)

HIGH

- **Audit log via Postgres triggers** — currently scattered `recordAudit()` calls. Move to AFTER INSERT/UPDATE/DELETE triggers on audited tables. Then remove application-level calls. Bundled with admin audit-log viewer UI.
- **Embedding-based semantic matcher** — replace LLM step 6 in matcher with pgvector cosine similarity. ~100× cheaper, semantically richer. Needs embedding column on sessions + tweets + a generation cron.
- **Server-side admin mutations** — `apiFeedService.addSource` et al. still write through browser client with RLS as sole defense. Migrate to server functions with `assertEditor` / `assertAdmin` + `supabaseAdmin` writes.

MEDIUM

- Per-IP rate limit on cron routes (defense in depth)
- Prompt-injection delimiters on `ai-summarize` edge function (matches what summarize-job already does)
- React.memo on TweetCard (currently re-renders all visible cards per poll)
- OnboardingWizard split (1530 LOC → step components)
- `media_urls` → JSONB with type metadata (video play-icon overlay)
- CSP header (none configured)

LOW

- Mock data files (~225KB) still bundled
- Some `as any` / `as never` casts in `apiFeedService`
- `routeTree.gen.ts` committed (should be gitignored)

NEAR-TERM FEATURE ROADMAP

1. **Bookmarks with notes** — universal save-for-later. Simple `user_bookmarks` table + button on every tweet card + `/me/saved` page.
2. **Digest preview** — generate sample digest from current filters before saving. Closes the "configure → wait 7 days → realize it's noise" iteration loop.
3. **AI Q&A over corpus** — natural-language search box in topbar. FTS + recency retrieval → LLM synthesis with citations.
4. **Historical backfill** — pluggable adapter for socialdata.tools / Apify / twitterapi.io. Lets users populate sessions for past congresses with prior tweet data.
5. **Web push notifications** — activate watchlists. Currently in-app only.

DECISIONS LOGGED ELSEWHERE

Architectural decision records live in commit messages and the handoff document (`urofeed_state_handoff.md`). Key non-obvious choices: TanStack Start over Vite, Lovable AI Gateway over per-user LLM keys, global tweets table over per-tenant, vault-only secret, hashtag-first matcher cascade, polymorphic summary `target_id`, `parent_handle/parent_text` snapshot on tweets, dark mode only.

WHERE TO START AS A NEW ENGINEER

1. Read `src/services/feedService.ts` — the contract every component uses.
2. Read `src/server/ingestion.server.ts` — the ingestion entry point.

3. Read `src/routes/api/public/hooks/match-tweets-to-sessions.ts` — the 7-step matcher.
4. Read `src/components/sessions/SessionDetail.tsx` — the crown jewel UI surface.
5. Read `src/components/wizard/OnboardingWizard.tsx` — most complex form flow.
6. Run the demo account locally and click every primary surface to learn the product before changing anything.